

Aplicativo Chat Completo

Para implementar a funcionalidade de mostrar os usuários logados no servidor para o cliente, precisamos modificar tanto o código do servidor quanto o do cliente.

Passos para a implementação:

No Servidor:

1. O servidor deve enviar para o cliente a lista de usuários logados sempre que um novo cliente se conectar ou se um cliente solicitar essa informação.

No Cliente:

1. O cliente pode ter um botão ou uma opção para pedir a lista de usuários logados.
2. O cliente deve ser capaz de exibir essa lista na interface.

Modificação no Código do Servidor:

No código do servidor, precisamos adicionar uma funcionalidade para notificar os clientes sobre a lista de usuários conectados. Uma boa forma de fazer isso é ao enviar a lista de clientes sempre que um cliente novo se conectar ou solicitar.

Aqui está como você pode modificar a classe `ClientHandler` para enviar a lista de usuários para o cliente:

// Dentro da classe `ClientHandler`, logo após o cliente se conectar:

// Enviar lista de usuários para o cliente

```
sendUserList();
```

// Função para enviar a lista de usuários conectados ao cliente

```
private void sendUserList() {  
    synchronized (clients) {  
        StringBuilder userList = new StringBuilder("Usuários conectados:\n");  
        for (String client : clients.keySet()) {  
            userList.append(client).append("\n");  
        }  
        out.println(userList.toString());  
    }  
}
```

Esse código irá enviar a lista de usuários conectados ao cliente assim que ele se conectar. Você também pode chamar `sendUserList()` em outros momentos, como quando um cliente solicitar a lista.

Modificação no Código do Cliente:

No lado do cliente, você pode adicionar uma funcionalidade para exibir os usuários logados na interface gráfica.

1. **Adicionar um botão para solicitar a lista de usuários.**
2. **Mostrar a lista na área de texto (ou em outro componente da interface).**

Aqui está como você pode fazer isso:

1. **Adicionar um botão para solicitar a lista de usuários:**

```
private JButton btnShowUsers; // Novo botão para mostrar usuários
```

2. **Adicionar o botão na interface:**

No método `AppChatClientV2()` (onde a interface gráfica é configurada), adicione um novo painel para o botão que solicitará a lista de usuários:

```
// Painel para o botão de mostrar usuários
```

```
btnShowUsers = new JButton("Mostrar Usuários");
```

```
panel.add(btnShowUsers, BorderLayout.WEST);
```

```
// Ação do botão de mostrar usuários
```

```
btnShowUsers.addActionListener(new ActionListener() {
```

```
    public void actionPerformed(ActionEvent e) {
```

```
        showUserList();
```

```
    }
```

```
});
```

3. **Função para enviar o comando de solicitação da lista de usuários:**

```
private void showUserList() {
```

```
    out.println("/list"); // Comando para solicitar a lista de usuários
```

```
}
```

4. **Modifique o `MessageReceiver` para processar a lista de usuários:**

No `MessageReceiver`, você precisará adicionar a lógica para tratar a resposta do servidor com a lista de usuários. Isso pode ser feito dentro do loop `while`.

```
private class MessageReceiver implements Runnable {
```

```
    public void run() {
```

```

try {
    String message;
    while ((message = in.readLine()) != null) {
        if (message.startsWith("Usuários conectados:")) {
            taChat.append(message + "\n");
        } else {
            taChat.append(message + "\n");
        }
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

Conclusão:

Com essas modificações, sempre que o cliente clicar no botão "Mostrar Usuários", ele irá enviar um comando para o servidor ("/list"). O servidor então irá responder com a lista de usuários conectados, que será exibida no cliente.

Resumo das mudanças:

- **Servidor:** Enviar a lista de usuários conectados para os clientes.
- **Cliente:** Adicionar um botão para solicitar a lista de usuários e exibir essa lista na interface.

Essas modificações irão adicionar a funcionalidade desejada de mostrar os usuários logados no servidor.

Claro! Vou apresentar os códigos completos do **Cliente** e **Servidor** com as alterações que implementam a funcionalidade de mostrar os usuários logados.

Código Alterado do Cliente (AppChatClientV2):

```
import java.awt.*;

import java.awt.event.*;

import java.io.*;

import java.net.*;

import javax.swing.*;

public class AppChatClientV2 extends JFrame {

    private JTextArea taChat;

    private JTextField tfMessage;

    private JTextField tfRecipient;

    private JButton btnSend;

    private JButton btnShowUsers; // Novo botão para mostrar usuários

    private Socket socket;

    private PrintWriter out;

    private BufferedReader in;

    private String serverAddress = "192.168.1.10"; // Endereço do servidor

    private int port = 12345; // Porta do servidor

    public AppChatClientV2() {

        // Configurações da janela

        setTitle("Chat Client");

        setSize(400, 300);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setLayout(new BorderLayout());

        taChat = new JTextArea();

        taChat.setEditable(false);

        add(new JScrollPane(taChat), BorderLayout.CENTER);
```

```
JPanel panel = new JPanel();  
panel.setLayout(new BorderLayout());  
  
// Adicionando o label e o campo de texto para o destinatário  
JPanel recipientPanel = new JPanel();  
recipientPanel.setLayout(new FlowLayout(FlowLayout.LEFT));  
JLabel lblRecipient = new JLabel("Destinatário:");  
recipientPanel.add(lblRecipient);  
JTextField tfRecipient = new JTextField(15);  
recipientPanel.add(tfRecipient);  
panel.add(recipientPanel, BorderLayout.NORTH);  
  
// Adicionando o label e o campo de texto para a mensagem  
JPanel messagePanel = new JPanel();  
messagePanel.setLayout(new FlowLayout(FlowLayout.LEFT));  
JLabel lblMessage = new JLabel("Mensagem:");  
messagePanel.add(lblMessage);  
JTextField tfMessage = new JTextField(20);  
messagePanel.add(tfMessage);  
panel.add(messagePanel, BorderLayout.CENTER);  
  
// Botão de envio  
JButton btnSend = new JButton("Enviar");  
panel.add(btnSend, BorderLayout.EAST);  
  
// Novo botão para mostrar usuários conectados  
JButton btnShowUsers = new JButton("Mostrar Usuários");  
panel.add(btnShowUsers, BorderLayout.WEST);  
  
add(panel, BorderLayout.SOUTH);
```

```

// Ação do botão de enviar
btnSend.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        sendMessage();
    }
});

// Ação do botão de mostrar usuários
btnShowUsers.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        showUserList();
    }
});

// Conectar ao servidor
connectToServer();

// Iniciar a thread de recebimento de mensagens
new Thread(new MessageReceiver()).start();
}

private void connectToServer() {
    try {
        // Selecionar o Servidor
        String iphost = JOptionPane.showInputDialog("Digite o ip do servidor:");
        serverAddress = iphost;
        socket = new Socket(serverAddress, port);
        out = new PrintWriter(socket.getOutputStream(), true);
        in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
    }
}

```

```

        // Enviar o nome do cliente

        String name = JOptionPane.showInputDialog("Digite seu nome:");

        setTitle("Chat Client - " + name);

        out.println(name);
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private void sendMessage() {
    String recipient = tfRecipient.getText();
    String message = tfMessage.getText();

    if (!message.isEmpty() && !recipient.isEmpty()) {
        out.println("/send " + recipient + " " + message); // Envia a mensagem para o servidor
        taChat.append("Você (para " + recipient + "): " + message + "\n");
        tfMessage.setText("");
    }
}

private void showUserList() {
    out.println("/list"); // Envia o comando para solicitar a lista de usuários
}

private class MessageReceiver implements Runnable {
    public void run() {
        try {
            String message;

            while ((message = in.readLine()) != null) {
                if (message.startsWith("Usuários conectados:")) {
                    taChat.append(message + "\n");
                } else {

```

```

        taChat.append(message + "\n");
    }
}
} catch (IOException e) {
    e.printStackTrace();
}
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new AppChatClientV2().setVisible(true);
        }
    });
}
}

```

Alterações no Cliente:

1. **Novo botão** para mostrar os usuários conectados (btnShowUsers).
2. Ação de **mostrar a lista de usuários** com o comando /list sendo enviado para o servidor.
3. Modificação na classe MessageReceiver para processar a resposta do servidor sobre os usuários conectados, exibindo a lista na interface do cliente.

Código Alterado do Servidor (AppChatServerV2):

```

import java.io.*;
import java.net.*;
import java.util.*;

public class AppChatServerV2 {
    private static final int PORT = 12345; // Alterado para a mesma porta do cliente
    private static ServerSocket serverSocket;

```



```
private static Map<String, PrintWriter> clients = new HashMap<>();
```

```
public static void main(String[] args) {
```

```
    try {
```

```
        serverSocket = new ServerSocket(PORT);
```

```
        System.out.println("Servidor aguardando conexões...");
```

```
        while (true) {
```

```
            Socket clientSocket = serverSocket.accept();
```

```
            System.out.println("Cliente conectado: " + clientSocket.getInetAddress());
```

```
            // Criação de nova thread para cada cliente conectado
```

```
            new Thread(new ClientHandler(clientSocket)).start();
```

```
        }
```

```
    } catch (IOException e) {
```

```
        e.printStackTrace();
```

```
    }
```

```
}
```

```
// Classe que gerencia a comunicação com o cliente
```

```
private static class ClientHandler implements Runnable {
```

```
    private Socket socket;
```

```
    private BufferedReader in;
```

```
    private PrintWriter out;
```

```
    private String clientName;
```

```
    public ClientHandler(Socket socket) {
```

```
        this.socket = socket;
```

```
        try {
```

```
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
```

```
            out = new PrintWriter(socket.getOutputStream(), true);
```

```
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

```
public void run() {  
    try {  
        // Receber o nome do cliente  
        out.println("Digite seu nome:");  
        clientName = in.readLine();  
        synchronized (clients) {  
            clients.put(clientName, out);  
        }  
        System.out.println(clientName + " entrou no chat.");  
  
        // Enviar a lista de usuários conectados ao cliente  
        sendUserList(); // Envia a lista de usuários após a conexão  
  
        // Enviar mensagens do cliente para o servidor  
        String message;  
        while ((message = in.readLine()) != null) {  
            if (message.startsWith("/send")) {  
                // Comando para enviar mensagem para outro cliente  
                String[] parts = message.split(" ", 3);  
                if (parts.length == 3) {  
                    String target = parts[1];  
                    String msg = parts[2];  
                    sendMessageToClient(target, msg);  
                }  
            } else if (message.startsWith("/list")) {  
                // Comando para enviar a lista de usuários conectados
```

```

        sendUserList();
    } else {
        System.out.println(clientName + ": " + message);
    }
}
} catch (IOException e) {
    e.printStackTrace();
} finally {
    // Remover cliente da lista ao desconectar
    synchronized (clients) {
        clients.remove(clientName);
    }
    try {
        socket.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
}

```

```

private void sendMessageToClient(String target, String message) {
    PrintWriter targetOut = clients.get(target);
    if (targetOut != null) {
        targetOut.println(clientName + " diz: " + message);
    } else {
        out.println("Usuário " + target + " não encontrado.");
    }
}
}

```

```

private void sendUserList() {
    synchronized (clients) {

```

```
StringBuilder userList = new StringBuilder("Usuários conectados:\n");  
  
for (String client : clients.keySet()) {  
    userList.append(client).append("\n");  
}  
  
out.println(userList.toString());  
}  
}  
}
```

Alterações no Servidor:

1. **Envio da lista de usuários** (sendUserList) para o cliente sempre que ele se conecta ou solicita com o comando /list.
2. Modificação no comando de recebimento para tratar o comando /list enviado pelo cliente, retornando a lista de usuários conectados.

Como funciona:

- **No servidor:** Quando um cliente se conecta, ele recebe a lista de todos os usuários conectados no momento. Sempre que um cliente envia o comando /list, o servidor retorna a lista atualizada de usuários conectados.
- **No cliente:** O cliente pode clicar no botão "Mostrar Usuários" para enviar o comando /list ao servidor e receber a lista de usuários conectados. Essa lista é então exibida na área de texto.

Com essas modificações, o sistema de chat agora suporta a visualização dos usuários logados no servidor, tanto para o cliente quanto para o servidor.

Que ótimo que você gostou! Para melhorar a funcionalidade conforme o seu pedido, vamos fazer as seguintes alterações:

1. **Listar os usuários em um JList (ou JListBox) no cliente**, ao invés de exibir na JTextArea.
2. **Exibir a lista de usuários automaticamente**, sem precisar clicar em um botão, sempre que um usuário se conecta ou a lista é atualizada.

Alterações no código do Cliente:

Vamos substituir a exibição da lista de usuários na JTextArea por um JList que será atualizado automaticamente.

Código Alterado do Cliente (com JList para mostrar usuários):

```
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;
import javax.swing.*;
import java.util.*;

public class AppChatClientV2 extends JFrame {

    private JTextArea taChat;
    private JTextField tfMessage;
    private JTextField tfRecipient;
    private JButton btnSend;
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;
    private String serverAddress = "192.168.1.10"; // Endereço do servidor
    private int port = 12345; // Porta do servidor
    private DefaultListModel<String> userListModel; // Modelo da JList para usuários
    private JList<String> userList; // JList para mostrar usuários conectados

    public AppChatClientV2() {
        // Configurações da janela
        setTitle("Chat Client");
```

```
setSize(600, 400); // Aumentar o tamanho para caber a JList
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLayout(new BorderLayout());

// Configuração da área de chat
taChat = new JTextArea();
taChat.setEditable(false);
add(new JScrollPane(taChat), BorderLayout.CENTER);

// Paineira para campos de mensagem e destinatário
JPanel panel = new JPanel();
panel.setLayout(new BorderLayout());

JPanel recipientPanel = new JPanel();
recipientPanel.setLayout(new FlowLayout(FlowLayout.LEFT));
JLabel lblRecipient = new JLabel("Destinatário:");
recipientPanel.add(lblRecipient);
tfRecipient = new JTextField(15);
recipientPanel.add(tfRecipient);
panel.add(recipientPanel, BorderLayout.NORTH);

JPanel messagePanel = new JPanel();
messagePanel.setLayout(new FlowLayout(FlowLayout.LEFT));
JLabel lblMessage = new JLabel("Mensagem:");
messagePanel.add(lblMessage);
tfMessage = new JTextField(20);
messagePanel.add(tfMessage);
panel.add(messagePanel, BorderLayout.CENTER);

btnSend = new JButton("Enviar");
panel.add(btnSend, BorderLayout.EAST);
```

```

add(panel, BorderLayout.SOUTH);

// Novo painel para mostrar a lista de usuários conectados
JPanel userPanel = new JPanel();
userPanel.setLayout(new BorderLayout());
userPanel.setBorder(BorderFactory.createTitledBorder("Usuários Conectados"));

// Criar e configurar o JList para mostrar os usuários
userListModel = new DefaultListModel<>();
userList = new JList<>(userListModel);
userList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
userPanel.add(new JScrollPane(userList), BorderLayout.CENTER);

add(userPanel, BorderLayout.WEST); // Adicionando o painel de usuários à esquerda

// Ação do botão de enviar
btnSend.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        sendMessage();
    }
});

// Conectar ao servidor
connectToServer();

// Iniciar a thread de recebimento de mensagens
new Thread(new MessageReceiver()).start();
}

private void connectToServer() {

```

```

try {

    // Selecionar o Servidor

    String iphost = JOptionPane.showInputDialog("Digite o ip do servidor:");

    serverAddress = iphost;

    socket = new Socket(serverAddress, port);

    out = new PrintWriter(socket.getOutputStream(), true);

    in = new BufferedReader(new InputStreamReader(socket.getInputStream()));


    // Enviar o nome do cliente

    String name = JOptionPane.showInputDialog("Digite seu nome:");

    setTitle("Chat Client - " + name);

    out.println(name);
} catch (IOException e) {

    e.printStackTrace();

}

}

private void sendMessage() {

    String recipient = tfRecipient.getText();

    String message = tfMessage.getText();

    if (!message.isEmpty() && !recipient.isEmpty()) {

        out.println("/send " + recipient + " " + message); // Envia a mensagem para o servidor

        taChat.append("Você (para " + recipient + "): " + message + "\n");

        tfMessage.setText("");

    }

}

private class MessageReceiver implements Runnable {

    public void run() {

        try {

            String message;

```



```

while ((message = in.readLine()) != null) {
    if (message.startsWith("Usuários conectados:")) {
        // Receber e atualizar a lista de usuários
        updateUserList(message);
    } else {
        taChat.append(message + "\n");
    }
}
} catch (IOException e) {
    e.printStackTrace();
}
}

// Método para atualizar a JList de usuários
private void updateUserList(String message) {
    String[] users = message.split("\n");
    userListModel.clear(); // Limpa a lista atual
    for (String user : users) {
        if (!user.isEmpty()) {
            userListModel.addElement(user); // Adiciona os usuários à JList
        }
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new AppChatClientV2().setVisible(true);
        }
    });
}

```

```
}  
  
}
```

Alterações no Cliente:

1. **Uso de JList:** Substituímos o uso de JTextArea para a exibição de usuários por um JList, que será atualizado automaticamente.
2. **Atualização automática da lista de usuários:** Agora, o cliente recebe a lista de usuários conectados sempre que o servidor envia uma mensagem sobre os usuários. Essa lista é atualizada automaticamente no JList.
3. **Ajuste no layout:** Adicionamos um painel (userPanel) à esquerda da interface, onde a JList é exibida.

Código Alterado do Servidor (AppChatServerV2):

No lado do servidor, as alterações principais são enviar a lista de usuários automaticamente sempre que um novo usuário se conecta.

```
import java.io.*;
```

```
import java.net.*;
```

```
import java.util.*;
```

```
public class AppChatServerV2 {
```

```
    private static final int PORT = 12345; // Porta do servidor
```

```
    private static ServerSocket serverSocket;
```

```
    private static Map<String, PrintWriter> clients = new HashMap<>();
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            serverSocket = new ServerSocket(PORT);
```

```
            System.out.println("Servidor aguardando conexões...");
```

```
            while (true) {
```

```
                Socket clientSocket = serverSocket.accept();
```

```
                System.out.println("Cliente conectado: " + clientSocket.getInetAddress());
```

```
                // Criação de nova thread para cada cliente conectado
```

```

        new Thread(new ClientHandler(clientSocket)).start();
    }
} catch (IOException e) {
    e.printStackTrace();
}
}

```

// Classe que gerencia a comunicação com o cliente

```

private static class ClientHandler implements Runnable {

    private Socket socket;

    private BufferedReader in;

    private PrintWriter out;

    private String clientName;

    public ClientHandler(Socket socket) {

        this.socket = socket;

        try {

            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

            out = new PrintWriter(socket.getOutputStream(), true);

        } catch (IOException e) {

            e.printStackTrace();

        }

    }

    public void run() {

        try {

            // Receber o nome do cliente

            out.println("Digite seu nome:");

            clientName = in.readLine();

            synchronized (clients) {

                clients.put(clientName, out);

            }

        }

    }

}

```

```

    }

    System.out.println(clientName + " entrou no chat.");

    // Enviar a lista de usuários conectados ao cliente
    sendUserList(); // Envia a lista de usuários após a conexão

    // Enviar mensagens do cliente para o servidor
    String message;
    while ((message = in.readLine()) != null) {
        if (message.startsWith("/send")) {
            // Comando para enviar mensagem para outro cliente
            String[] parts = message.split(" ", 3);
            if (parts.length == 3) {
                String target = parts[1];
                String msg = parts[2];
                sendMessageToClient(target, msg);
            }
        } else if (message.startsWith("/list")) {
            // Comando para enviar a lista de usuários conectados
            sendUserList();
        } else {
            System.out.println(clientName + ": " + message);
        }
    }
} catch (IOException e) {
    e.printStackTrace();
} finally {
    // Remover cliente da lista ao desconectar
    synchronized (clients) {
        clients.remove(clientName);
    }
}

```

```

        try {
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

private void sendMessageToClient(String target, String message) {
    PrintWriter targetOut = clients.get(target);
    if (targetOut != null) {
        targetOut.println(clientName + " diz: " + message);
    } else {
        out.println("Usuário " + target + " não encontrado.");
    }
}

private void sendUserList() {
    synchronized (clients) {
        StringBuilder userList = new StringBuilder("Usuários conectados:\n");
        for (String client : clients.keySet()) {
            userList.append(client).append("\n");
        }
        out.println(userList.toString());
    }
}
}

```

Alterações no Servidor:

1. O servidor agora envia automaticamente a lista de usuários conectados para todos os clientes sempre que um novo cliente se conecta.

2. O servidor também envia a lista de usuários sempre que o comando /list é recebido.

Agora, com essas modificações, a lista de usuários será exibida automaticamente em uma JList no cliente assim que o cliente se conectar, e será atualizada automaticamente sempre que a lista de usuários mudar. Não há mais necessidade de clicar em um botão para solicitar a lista de usuários!

Versão 4

Entendido! Vamos simplificar a abordagem. Ao invés de enviar múltiplas mensagens ou tentar atualizar a lista de usuários diretamente a cada nova conexão, enviaremos uma única mensagem com todos os usuários conectados no momento, separados por um caractere delimitador, como uma vírgula. Assim, o cliente poderá processar essa mensagem e atualizar o JList.

Estratégia:

1. **Servidor:** O servidor irá enviar uma única mensagem com todos os nomes de usuários conectados, separados por um delimitador (por exemplo, vírgula ,), a cada vez que um cliente se conecta ou desconecta.
2. **Cliente:** O cliente vai receber essa lista e extrair os nomes dos usuários, exibindo-os no JList.

Alterações no código:

- **Servidor:** Quando um cliente se conecta ou desconecta, o servidor envia a lista atualizada de usuários para todos os clientes conectados.
- **Cliente:** O cliente processa a mensagem recebida para atualizar a lista de usuários no JList.

Vamos começar com as mudanças necessárias:

Servidor Atualizado (com envio único de lista de usuários):

```
import java.io.*;
import java.net.*;
import java.util.*;

public class AppChatServerV2 {
    private static final int PORT = 12345;
    private static ServerSocket serverSocket;
    private static Set<String> clients = new HashSet<>();
    private static Map<String, PrintWriter> clientOutputs = new HashMap<>();

    public static void main(String[] args) {
        try {
            serverSocket = new ServerSocket(PORT);
            System.out.println("Servidor aguardando conexões...");
```

```

while (true) {
    Socket clientSocket = serverSocket.accept();

    System.out.println("Cliente conectado: " + clientSocket.getInetAddress());

    // Criação de nova thread para cada cliente conectado
    new Thread(new ClientHandler(clientSocket)).start();
}
} catch (IOException e) {
    e.printStackTrace();
}
}

private static class ClientHandler implements Runnable {
    private Socket socket;
    private BufferedReader in;
    private PrintWriter out;
    private String clientName;

    public ClientHandler(Socket socket) {
        this.socket = socket;

        try {
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public void run() {
        try {

```



```

// Receber o nome do cliente
out.println("Digite seu nome:");
clientName = in.readLine();
synchronized (clients) {
    clients.add(clientName);
    clientOutputs.put(clientName, out);
}
System.out.println(clientName + " entrou no chat.");

// Enviar a lista de usuários para o novo cliente
sendUserListToAllClients();

// Enviar mensagens do cliente para o servidor
String message;
while ((message = in.readLine()) != null) {
    if (message.startsWith("/send")) {
        // Comando para enviar mensagem para outro cliente
        String[] parts = message.split(" ", 3);
        if (parts.length == 3) {
            String target = parts[1];
            String msg = parts[2];
            sendMessageToClient(target, msg);
        }
    } else {
        System.out.println(clientName + ": " + message);
    }
}
} catch (IOException e) {
    e.printStackTrace();
} finally {
    // Remover cliente da lista ao desconectar

```

```

        synchronized (clients) {
            clients.remove(clientName);
            clientOutputs.remove(clientName);
        }

        // Enviar a lista de usuários atualizada para todos os clientes
        sendUserListToAllClients();
    }
    try {
        socket.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

```

private void sendMessageToClient(String target, String message) {
    PrintWriter targetOut = clientOutputs.get(target);
    if (targetOut != null) {
        targetOut.println(clientName + " diz: " + message);
    } else {
        out.println("Usuário " + target + " não encontrado.");
    }
}
}

```

```

private void sendUserListToAllClients() {
    synchronized (clients) {
        // Criar a lista de usuários conectados, separada por vírgula
        String userList = String.join(",", clients);
        String userListMessage = "/userlist " + userList;

        // Enviar a lista de usuários para todos os clientes
        for (PrintWriter clientOut : clientOutputs.values()) {

```

```

        clientOut.println(userListMessage);
    }
}
}
}
}
}
}

```

Cliente Atualizado (processando a lista de usuários):

No cliente, vamos receber a mensagem de usuários conectados e atualizar o JList adequadamente.

```

import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;
import javax.swing.*;
import java.util.*;

public class AppChatClientV2 extends JFrame {

    private JTextArea taChat;
    private JTextField tfMessage;
    private JTextField tfRecipient;
    private JButton btnSend;
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;
    private String serverAddress = "192.168.1.10"; // Endereço do servidor
    private int port = 12345; // Porta do servidor
    private DefaultListModel<String> userListModel; // Modelo para o JList
    private JList<String> userList;

    public AppChatClientV2() {

```

```
// Configurações da janela

setTitle("Chat Client");

setSize(500, 400);

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

setLayout(new BorderLayout());


taChat = new JTextArea();
taChat.setEditable(false);
add(new JScrollPane(taChat), BorderLayout.CENTER);


JPanel panel = new JPanel();
panel.setLayout(new BorderLayout());


// Adicionando o label e o campo de texto para o destinatário
JPanel recipientPanel = new JPanel();
recipientPanel.setLayout(new FlowLayout(FlowLayout.LEFT));
JLabel lblRecipient = new JLabel("Destinatário:");
recipientPanel.add(lblRecipient);
tfRecipient = new JTextField(15);
recipientPanel.add(tfRecipient);
panel.add(recipientPanel, BorderLayout.NORTH);


// Adicionando o label e o campo de texto para a mensagem
JPanel messagePanel = new JPanel();
messagePanel.setLayout(new FlowLayout(FlowLayout.LEFT));
JLabel lblMessage = new JLabel("Mensagem:");
messagePanel.add(lblMessage);
tfMessage = new JTextField(20);
messagePanel.add(tfMessage);
panel.add(messagePanel, BorderLayout.CENTER);
```

```

// Botão de envio

btnSend = new JButton("Enviar");

panel.add(btnSend, BorderLayout.EAST);


add(panel, BorderLayout.SOUTH);


// Criar o JList para mostrar os usuários conectados
userListModel = new DefaultListModel<>();

userList = new JList<>(userListModel);

JScrollPane scrollPane = new JScrollPane(userList);
add(scrollPane, BorderLayout.WEST);


// Ação do botão de enviar
btnSend.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        sendMessage();
    }
});


// Conectar ao servidor
connectToServer();


// Iniciar a thread de recebimento de mensagens
new Thread(new MessageReceiver()).start();
}

private void connectToServer() {
    try {
        // Selecionar o Servidor

        String iphost = JOptionPane.showInputDialog("Digite o ip do servidor:");

        serverAddress = iphost;
    }
}

```

```

        socket = new Socket(serverAddress, port);

        out = new PrintWriter(socket.getOutputStream(), true);

        in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

        // Enviar o nome do cliente

        String name = JOptionPane.showInputDialog("Digite seu nome:");

        setTitle("Chat Client - " + name);

        out.println(name);
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private void sendMessage() {
    String recipient = tfRecipient.getText();

    String message = tfMessage.getText();

    if (!message.isEmpty() && !recipient.isEmpty()) {
        out.println("/send " + recipient + " " + message); // Envia a mensagem para o servidor
        taChat.append("Você (para " + recipient + "): " + message + "\n");
        tfMessage.setText("");
    }
}

private class MessageReceiver implements Runnable {
    public void run() {
        try {
            String message;

            while ((message = in.readLine()) != null) {
                if (message.startsWith("/userlist")) {
                    // Atualiza o JList com a lista de usuários recebida
                    updateUserList(message);
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

```

        } else {
            taChat.append(message + "\n");
        }
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}

private void updateUserList(String message) {
    // Extrai os usuários da mensagem e atualiza o JList
    String userList = message.replace("/userlist ", "");
    String[] users = userList.split(",");
    userListModel.clear(); // Limpa a lista anterior
    for (String user : users) {
        userListModel.addElement(user);
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new AppChatClientV2().setVisible(true);
        }
    });
}
}

```

O que foi alterado:

1. **Servidor:** Agora, sempre que um cliente entra ou sai, o servidor envia a lista de todos os usuários conectados em uma única mensagem, separando-os por vírgulas.
2. **Cliente:** O cliente recebe essa lista e a processa para atualizar o JList com os nomes dos usuários. A lista de usuários é atualizada sempre que uma nova lista é recebida do servidor.

Como funciona:

- Quando um cliente entra ou sai, o servidor envia para todos os clientes conectados uma mensagem única com todos os usuários logados, separados por vírgula.
- O cliente processa essa mensagem e atualiza o JList com os nomes dos usuários.

Agora, essa abordagem deve resolver os problemas que você mencionou. Por favor, tente e veja se isso resolve a situação!

Perfeito! O que você deseja é que cada usuário seja mostrado em uma linha separada no JList, e não tudo em uma única linha. Vamos corrigir isso.

O que precisamos mudar:

- **Servidor:** Não precisa de mudanças, pois o servidor já envia a lista separada por vírgulas.
- **Cliente:** O que precisamos ajustar é o código que atualiza o JList para que cada nome de usuário apareça em uma linha separada.

Ao invés de adicionar todos os usuários como uma única string com vírgulas, vamos separar cada nome de usuário e adicioná-los individualmente ao JList.

Alteração no código do cliente:

Na função `updateUserList()`, vamos modificar o processo de adição dos usuários ao JList para que cada nome de usuário seja adicionado em uma nova linha.

Código atualizado do cliente:

```
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;
import javax.swing.*;
import java.util.*;

public class AppChatClientV2 extends JFrame {
    private JTextArea taChat;
    private JTextField tfMessage;
```



```
private JTextField tfRecipient;

private JButton btnSend;

private Socket socket;

private PrintWriter out;

private BufferedReader in;

private String serverAddress = "192.168.1.10"; // Endereço do servidor

private int port = 12345; // Porta do servidor

private DefaultListModel<String> userListModel; // Modelo para o JList

private JList<String> userList;
```

```
public AppChatClientV2() {

    // Configurações da janela

    setTitle("Chat Client");

    setSize(500, 400);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    setLayout(new BorderLayout());

    taChat = new JTextArea();

    taChat.setEditable(false);

    add(new JScrollPane(taChat), BorderLayout.CENTER);

    JPanel panel = new JPanel();

    panel.setLayout(new BorderLayout());

    // Adicionando o label e o campo de texto para o destinatário

    JPanel recipientPanel = new JPanel();

    recipientPanel.setLayout(new FlowLayout(FlowLayout.LEFT));

    JLabel lblRecipient = new JLabel("Destinatário:");

    recipientPanel.add(lblRecipient);

    tfRecipient = new JTextField(15);

    recipientPanel.add(tfRecipient);
```

```
panel.add(recipientPanel, BorderLayout.NORTH);

// Adicionando o label e o campo de texto para a mensagem
JPanel messagePanel = new JPanel();
messagePanel.setLayout(new FlowLayout(FlowLayout.LEFT));
JLabel lblMessage = new JLabel("Mensagem:");
messagePanel.add(lblMessage);
tfMessage = new JTextField(20);
messagePanel.add(tfMessage);
panel.add(messagePanel, BorderLayout.CENTER);

// Botão de envio
btnSend = new JButton("Enviar");
panel.add(btnSend, BorderLayout.EAST);

add(panel, BorderLayout.SOUTH);

// Criar o JList para mostrar os usuários conectados
userListModel = new DefaultListModel<>();
userList = new JList<>(userListModel);
JScrollPane scrollPane = new JScrollPane(userList);
add(scrollPane, BorderLayout.WEST);

// Ação do botão de enviar
btnSend.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        sendMessage();
    }
});

// Conectar ao servidor
```

```

connectToServer();

// Iniciar a thread de recebimento de mensagens
new Thread(new MessageReceiver()).start();
}

private void connectToServer() {
    try {
        // Selecionar o Servidor

        String iphost = JOptionPane.showInputDialog("Digite o ip do servidor:");
        serverAddress = iphost;
        socket = new Socket(serverAddress, port);
        out = new PrintWriter(socket.getOutputStream(), true);
        in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

        // Enviar o nome do cliente
        String name = JOptionPane.showInputDialog("Digite seu nome:");
        setTitle("Chat Client - " + name);
        out.println(name);
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private void sendMessage() {
    String recipient = tfRecipient.getText();
    String message = tfMessage.getText();
    if (!message.isEmpty() && !recipient.isEmpty()) {
        out.println("/send " + recipient + " " + message); // Envia a mensagem para o servidor
        taChat.append("Você (para " + recipient + "): " + message + "\n");
        tfMessage.setText("");
    }
}

```

```
}  
}
```

```
private class MessageReceiver implements Runnable {  
    public void run() {  
        try {  
            String message;  
            while ((message = in.readLine()) != null) {  
                if (message.startsWith("/userlist")) {  
                    // Atualiza o JList com a lista de usuários recebida  
                    updateUserList(message);  
                } else {  
                    taChat.append(message + "\n");  
                }  
            }  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
private void updateUserList(String message) {  
    // Extrai os usuários da mensagem e atualiza o JList  
    String userList = message.replace("/userlist ", "");  
    String[] users = userList.split(",");  
  
    // Limpa a lista de usuários existente  
    userListModel.clear();  
  
    // Adiciona cada usuário em uma linha do JList  
    for (String user : users) {
```

```

        userListModel.addElement(user);
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new AppChatClientV2().setVisible(true);
        }
    });
}
}

```

O que foi alterado:

1. **updateUserList:** Agora, em vez de adicionar todos os usuários como uma única string, estamos dividindo a string da lista de usuários, separando-os por vírgula, e adicionando cada usuário ao JList separadamente.
 - O método `split(",")` divide a string recebida pela vírgula, e então adicionamos cada nome individualmente ao `DefaultListModel`, que está vinculado ao JList.

Comportamento esperado:

- Quando o cliente receber a lista de usuários conectados, ele processará essa lista e atualizará o JList com um usuário por linha. Não haverá mais uma única linha contendo todos os nomes.

Agora, ao invés de todos os usuários aparecerem em uma linha, cada nome será exibido em uma linha separada no JList.

Por favor, tente essa versão do código e veja se ela resolve o problema!